

UNIVERSITÀ DEGLI STUDI DI
PERUGIA
Dipartimento di Matematica e Informatica



A.D. 1308
unipg
DIPARTIMENTO
DI MATEMATICA E INFORMATICA

INTERNSHIP

Use of CapeV2 sandbox for the analysis of multiple ransomware families

Academic Year 2024–2025

Contents

1	Introduction	3
2	Configuration and Usage	5
2.1	Platform	6
2.2	Platform Hypervisor	6
2.3	Sandbox Host/Host Machine	7
2.4	Sandbox Hypervisor	7
2.5	Sandbox Guest/Victim Machine	7
2.6	Configuration	7
2.7	Startup	8
2.8	Malware Submission	9
2.9	Analysis	11
2.9.1	Cape API	14
3	Overview of Malware	16
3.1	Current Landscape	16
3.2	Main Types of Malware	18
3.2.1	Viruses	18
3.2.2	Worms	18
3.2.3	Trojans	18
3.2.4	Keyloggers	18
3.2.5	Ransomware	19
4	Automated Ransomware Analysis	21

Chapter 1

Introduction

CapeV2 is an open-source malware sandbox that provides an automated analysis environment for malicious files. A sandbox is a tool used to execute suspicious or malicious files in an isolated environment while monitoring their behavior. CapeV2 is an extension of Cuckoo v1, which had the following features:

- Behavioral analysis of executables, based on the invoked system APIs;
- Capture of files created, modified, and/or deleted during execution;
- Network traffic capture, saved into a .pcap file;
- Malware classification based on signatures;
- Desktop screenshots during malware execution;
- Memory dump of the target system.

CapeV2 extends these features with:

- Malware detection using "YARA Rules", which scan files and memory to detect malicious activity;
- Payload extraction;
- A web interface to use the software.

CapeV2 is mainly designed to analyze:

- Generic Windows executables;
- DLL files;
- PDF documents;
- Microsoft Office documents;
- Other formats.

The software can be used standalone or integrated into other frameworks thanks to its modular design. The Cape sandbox consists of central software that manages the execution and analysis of malicious files. Each analysis is launched in an isolated virtual machine; the infrastructure consists of a host machine (which runs the central software) and one or more guest machines that host the execution of the malicious file. The following figure shows the architecture:

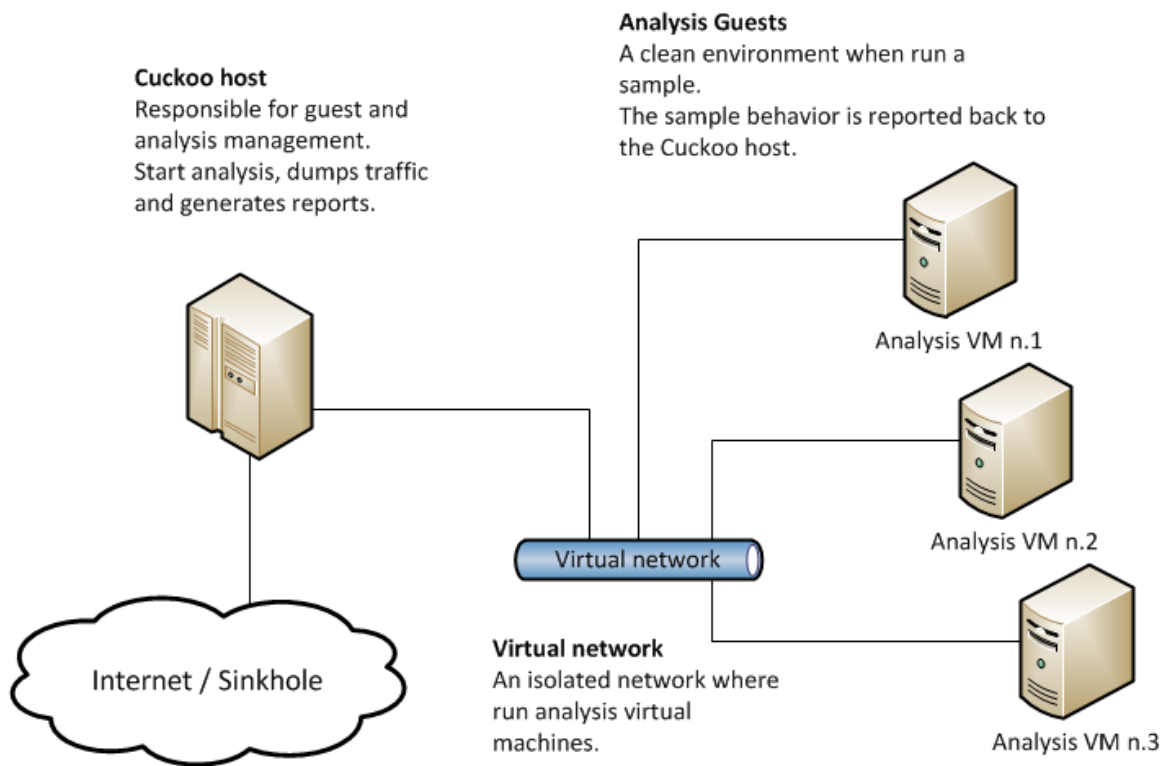


Figure 1.1: CapeV2 Architecture

Chapter 2

Configuration and Usage

There are several ways to configure CapeV2, but for security reasons "Nested Virtualization" is preferred. The setup starts from the host operating system where the platform hypervisor (virtual machine manager) is installed, inside of which a virtual machine is created to serve as the host machine or sandbox host. Inside the host machine, the sandbox hypervisor is installed, which in turn hosts a virtual machine acting as the sandbox guest or victim machine. Finally, an isolated network interface shared between the host machine and the victim machine is configured. Once configured, malware analysis can begin. Although CapeV2, according to the documentation, can run on any operating system, it was primarily designed for GNU/Linux distributions such as Ubuntu. As for the victim machine, CapeV2 is intended to analyze malware on Windows 10/11.

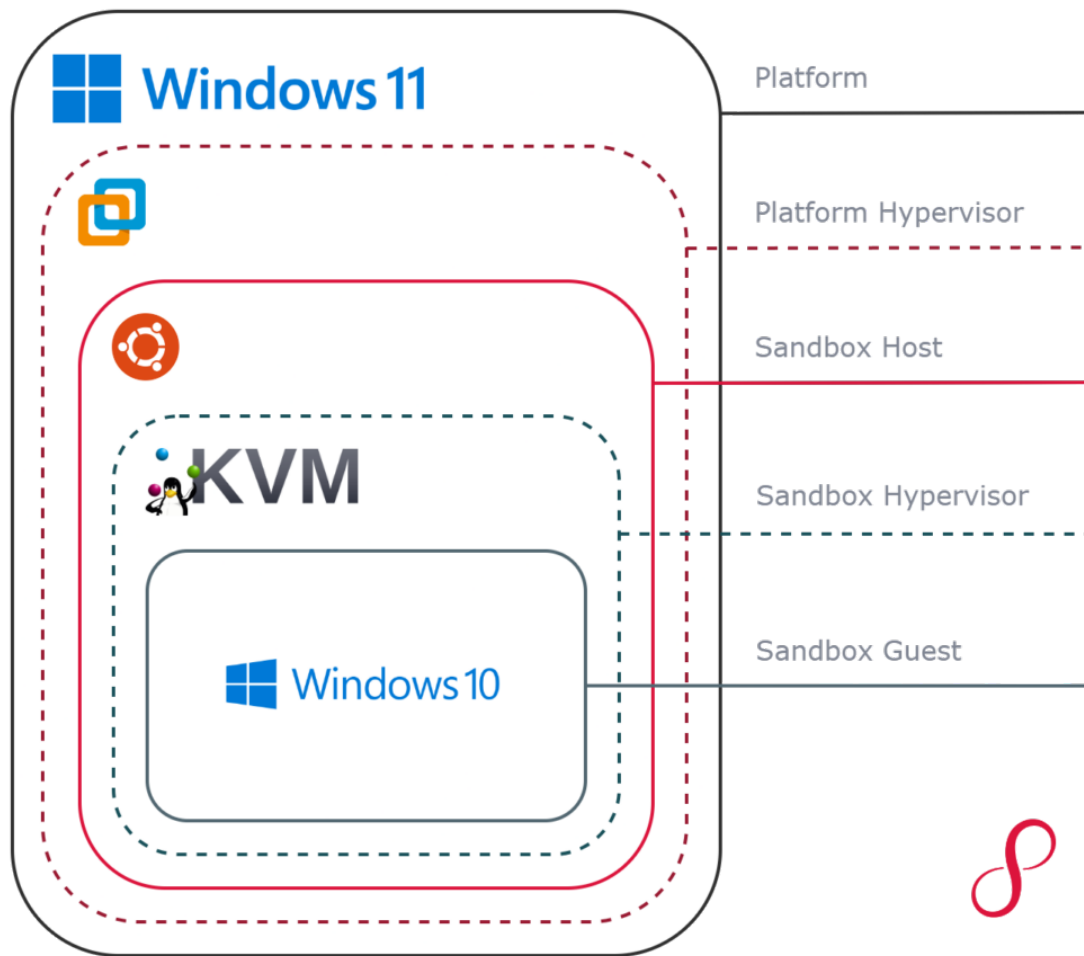


Figure 2.1: Nested Virtualization

2.1 Platform

The platform used in the setup is Windows 11.

2.2 Platform Hypervisor

The platform hypervisor installed on Windows 11 is VMware Workstation Pro.

2.3 Sandbox Host/Host Machine

Ubuntu LTS was chosen as the host operating system, where the CapeV2 installation script was subsequently downloaded and executed.

2.4 Sandbox Hypervisor

Virt-Manager was used as the sandbox hypervisor, leveraging the "libvirt" library to interface with KVM (Kernel-based Virtual Machine).

2.5 Sandbox Guest/Victim Machine

A Windows 10 virtual machine was downloaded and installed as the sandbox guest.

2.6 Configuration

1. Install all the software that a regular user typically runs on the victim machine; this prevents malware from detecting that it is running inside an analysis environment and partially or completely halting execution;
2. Disable Windows Defender, Real-time Protection, and the Firewall;
3. Download the Cape Agent, a Python program that transfers malware samples from the host machine to the victim machine. The agent's file extension must be renamed from .py to .pyw to allow silent execution, and a scheduled task must be created to launch the agent automatically when the victim machine boots;
4. Download Python;
5. The final step is to configure a network to enable communication between the host machine and the victim machine. An isolated network interface (host-only configuration) was set up on the host machine, allowing traffic exclusively between the victim machine and the host machine, with the IP subnet

192.168.100.0/24. The network adapter of the victim machine was assigned the IP address 192.168.100.50/24, and connectivity was tested using the "ping" command. Finally, a clean snapshot was taken so that the victim machine can be restored to a pristine state between malware analysis runs.

```
3: virbr1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:96:a1:8d brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.1/24 brd 192.168.100.255 scope global virbr1
        valid_lft forever preferred_lft forever
```

Figure 2.2: Isolated network interface on the host machine

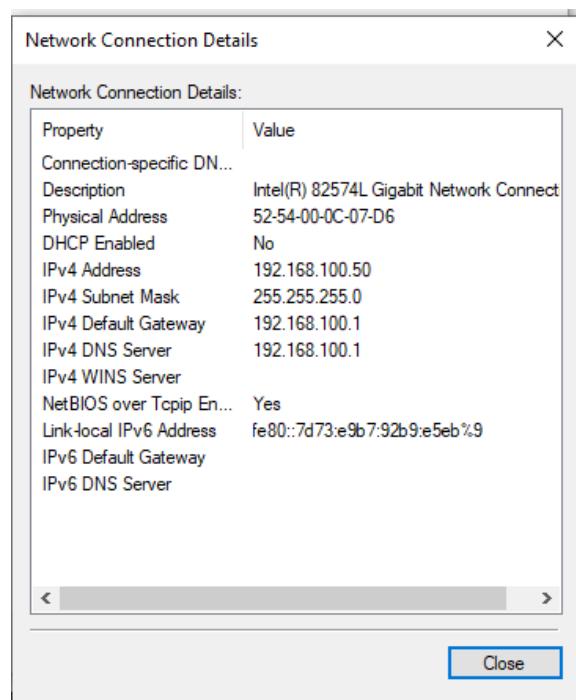


Figure 2.3: Isolated network interface on the victim machine

2.7 Startup

To start Cape, execute `cuckoo.py` located in `/opt/CAPEv2` using the following command:

```
sudo -u cape etc/poetry/bin/poetry run python opt/CAPEv2/cuckoo.py
```

Once running, open a browser and navigate to:

```
http://localhost:8000
```

This opens the CapeV2 web interface:

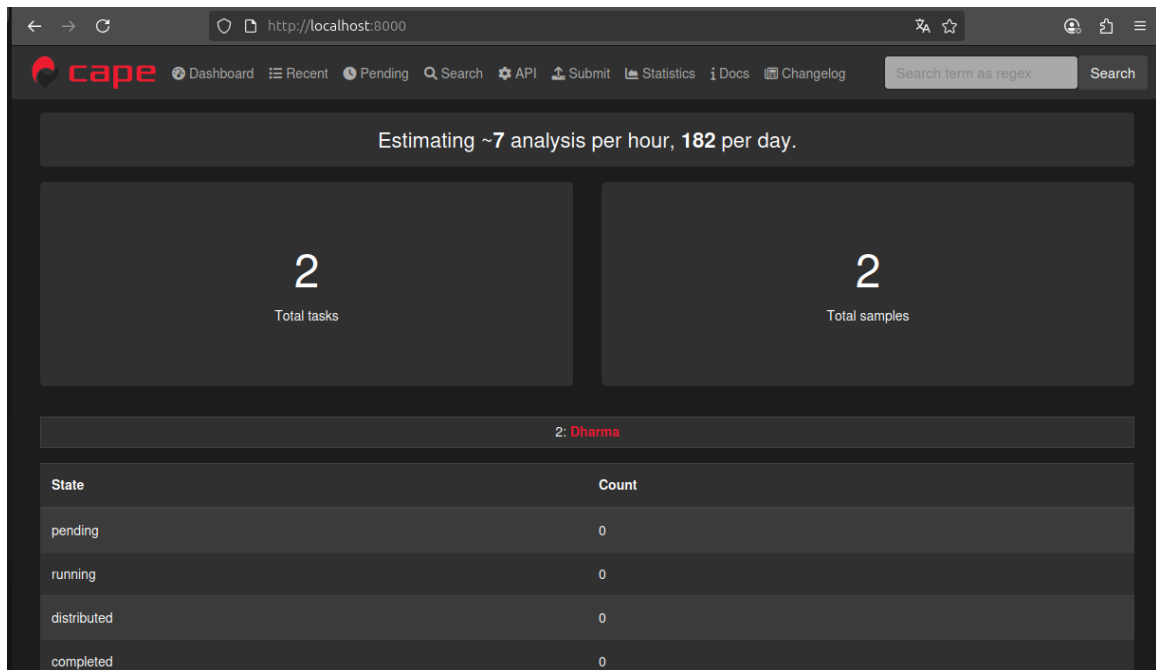


Figure 2.4: CapeV2 Web Interface

2.8 Malware Submission

To submit malware for analysis in Cape, a sample must first be downloaded. The primary platforms for retrieving samples include:

- `https://bazaar.abuse.ch/;`
- `https://virusShare.com;`
- `https://virusTotal.com;`

For example, to test the execution of MassLogger—a virus from the KeyLogger family that steals data by logging input devices—the process is as follows:

1. Download the MassLogger sample:



Figure 2.5: MassLogger sample

2. Go to the submit page in the Cape interface and select the MassLogger sample for analysis:

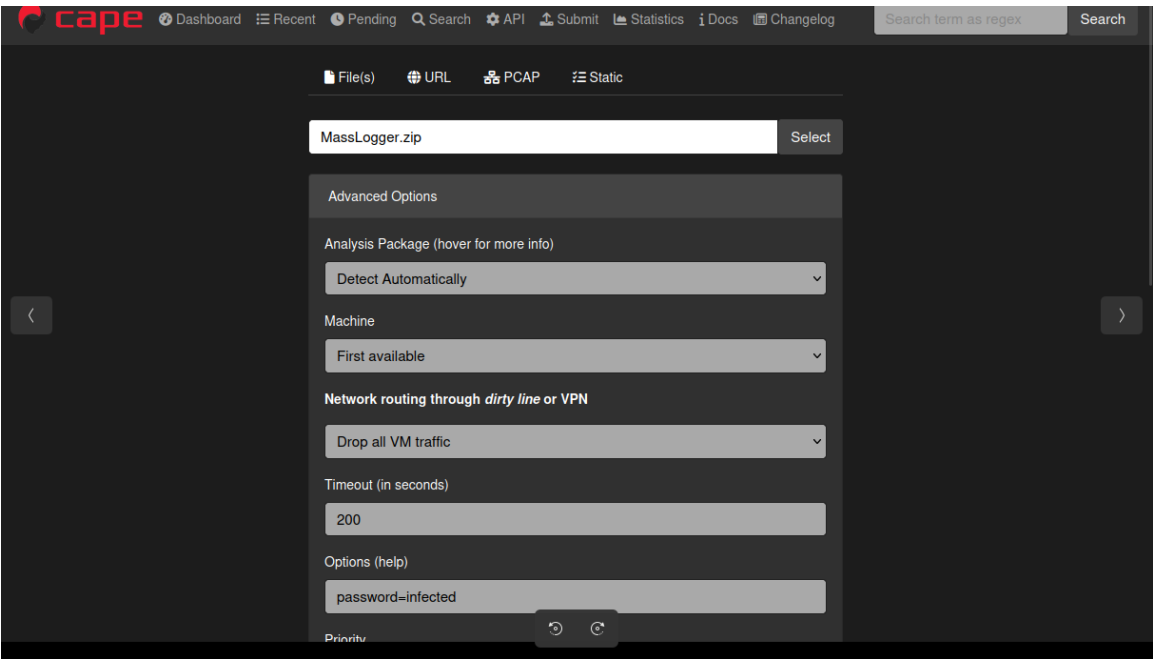


Figure 2.6: File submission

Once the file is submitted, Cape reverts the victim machine to its snapshot and starts the analysis.

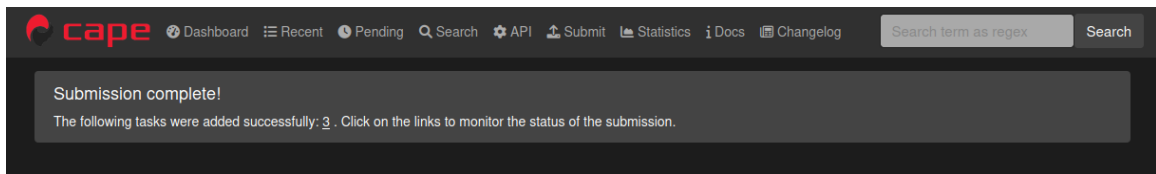


Figure 2.7: Submission completed

2.9 Analysis

Once malware execution is complete, several tools are available to analyze the sample, including:

- **Quick Overview:** includes analysis details and information about the analyzed file, screenshots taken on the victim machine during execution, signatures (unique patterns used by tools like Cape to detect malware) with the ability to inspect their details, accessed files, modified files, read files, deleted files, read registry keys, modified registry keys, resolved system APIs, executed commands, acquired mutexes, and started services.

Analysis

Category	Package	Started	Completed	Duration	Options	Log(s)
FILE		2025-07-26 12:38:51	2025-07-26 12:45:56	425 seconds	Show Options	Show Analysis Log

Machine

Name	Label	Manager	Started On	Shutdown On	Route
win10	win10	KVM	2025-07-26 12:38:51	2025-07-26 12:43:53	none

File Details

File Name	MassLogger.zip
File Type	Zip archive data, at least v5.1 to extract, compression method=AES Encrypted
File Size	600148 bytes
MD5	cd7a83ea9cad65bca50ef5c0aa0840f4
SHA1	27307e3566a8a18ef8e3e34a9740a7fe3de8a715

Figure 2.8: Quick Overview



Figure 2.9: Anomalous behavior detection via YARA rules



Figure 2.10: Screenshots captured during malware execution

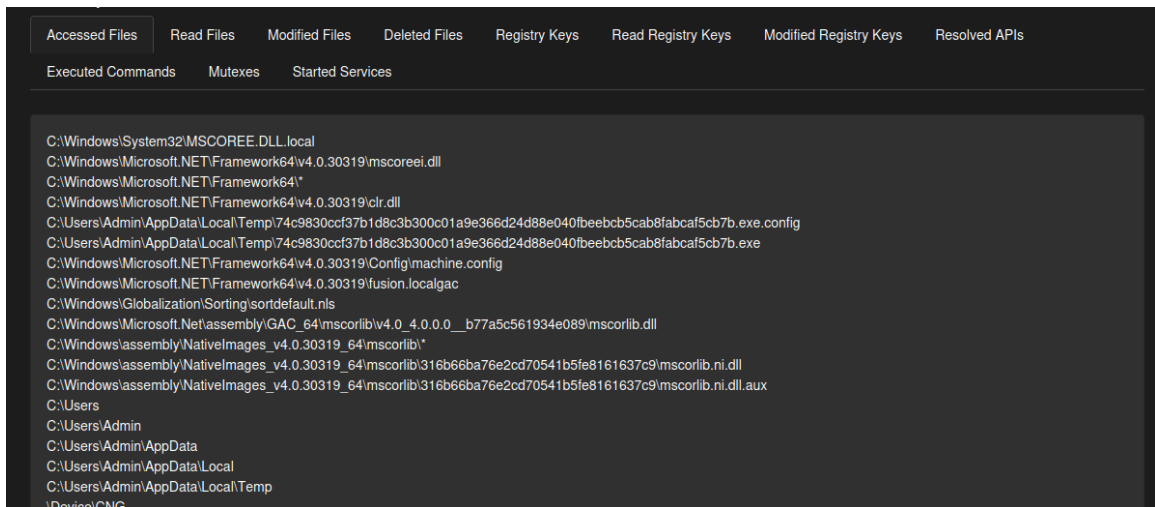


Figure 2.11: Accessed files

- Behavioral Analysis: includes the process tree, which allows inspecting the origin of processes along with their PID and PPID, and the invoked system APIs with details such as the calling thread, arguments, and return values. Filters can also be applied to the APIs.

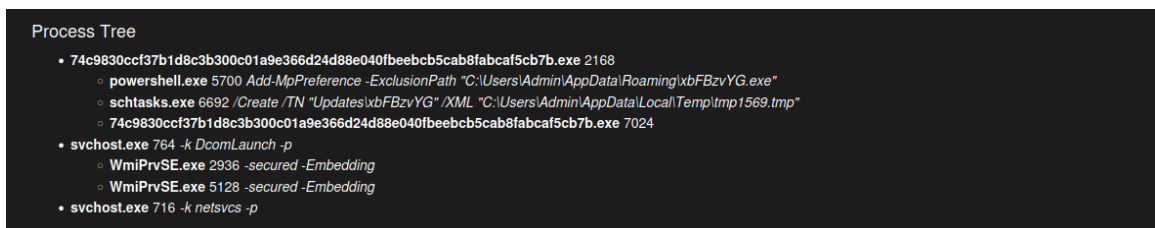


Figure 2.12: Process tree

Time	TID	Caller	API	Arguments	Status	Return	Repeated
2025-07-26 19:39:28,274	3 4 1 2	0x7ffb008 60ad 0x7ffb008 5c73	NtTestAlert		SUCCESS	0x00000000	
2025-07-26 19:39:28,274	1 1 6 0	0x7ffb001 e715 0x7ffb001 e37b	NtAllocateVirtualMemory	ProcessHandle: 0xffffffff BaseAddress: 0x010da000 RegionSize: 0x00001000 Protection: PAGE_READWRITE StackPivoted: no	SUCCESS	0x00000000	

Figure 2.13: Invoked system APIs

- **Dropped Files:** all files that the malware writes to disk during execution; these are usually used to perform secondary malicious actions. Cape allows downloading these files, submitting them for further analysis, or inspecting them directly.
- **Payloads:** extracted core components of the malware, representing the code sections responsible for harmful actions. Cape provides the same options for payloads as it does for dropped files.
- **.pcap Files:** during analysis, a network sniffer runs on the victim machine and generates a .pcap file that can be inspected with tools such as Wireshark or tcpdump.

2.9.1 Cape API

Cape provides an API to perform operations such as submitting files, deleting tasks, downloading malware payloads, and more. The list of available endpoints is accessible at <http://localhost:8000/apiv2/>. Through the `/opt/CAPEv2/conf/api.conf` configuration file, individual endpoints can be enabled or disabled. Since this is a RESTful API, the Django REST framework must first be installed via:

```
sudo -u cape etc/poetry/bin/poetry run pip install djangoRESTframework
```

There are two main ways to invoke the API:

1. From the command line (for example, submitting a file for analysis):

```
curl -F file=@/path/to/file http://localhost:8000/apiv2/tasks/create/file
```

2. Via a Python script (submitting a file for analysis):

```
1  import requests
2  import json
3
4  url = "http://localhost:8000/apiv2/tasks/create/file/"
5  SAMPLE_FILE = "/home/leonald/Scrivania/malware/Dharma.zip"
6
7  parameters = {
8      "options": "password=infected"
9  }
10
11  with open(SAMPLE_FILE,"rb") as sample:
12      multipart_file = {"file": ("temp_file_name", sample)}
13      response = requests.post(url, files=multipart_file,data = parameters)
14
15  if response.status_code == 200:
16      try:
17          data = response.json()
18          id_task = data['data']['task_ids'][0]
19          print(f"{response.status_code}")
20          print(id_task)
21      except json.JSONDecodeError:
22          print("not valid JSON")
23  else:
24      print("request error")
25      print(f"{response.status_code}")
26      print(f"{response.text}")
27
```

Figure 2.14: File submission with Python

Chapter 3

Overview of Malware

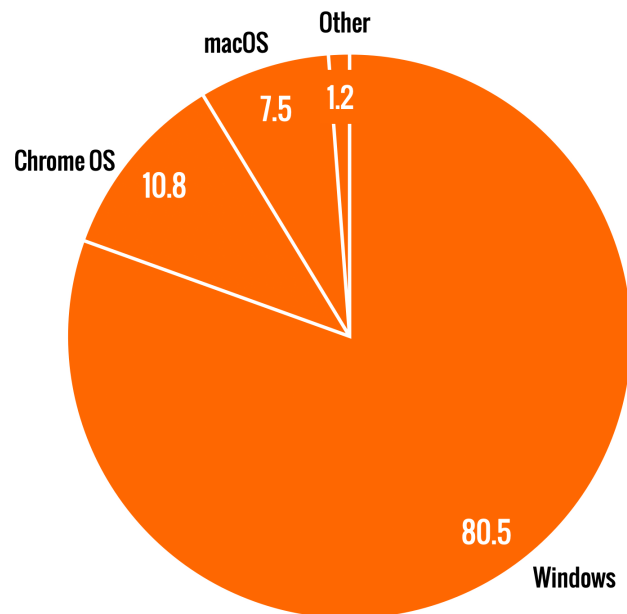
3.1 Current Landscape

Currently, approximately 560,000 new malware samples are detected every day, and over 1 billion variants are known to exist. In particular, around 5.33 vulnerabilities emerge every minute, meaning that cyber threats continue to pose a constant risk to all types of organizations worldwide. Cybersecurity therefore remains a critical concern for institutions and enterprises of every scale. Naturally, most malware is targeted at Windows systems, due to several key factors:

- Windows is the most widely adopted operating system globally; consequently, developers—often driven by financial incentives—prefer creating malware tailored specifically for Windows rather than other platforms;

OPERATING SYSTEM MARKET SHARE BY PLATFORM

Percent



Data source: IDC



Figure 3.1: Operating system market share

- Historically, Windows has presented a broader attack surface and more vulnerabilities, enabling malware to infect systems and spread more easily compared to other operating systems;
- Unlike operating systems such as macOS, Windows grants users greater freedom when downloading and installing third-party software; however, this flexibility is a double-edged sword, as unvetted applications may introduce vulnerabilities that malicious actors can exploit.

3.2 Main Types of Malware

3.2.1 Viruses

Viruses are the most common type of malware. They consist of self-replicating malicious code that spreads by modifying other host programs on the target machine and injecting malicious routines into them. They require user interaction to trigger execution, such as launching an infected executable or opening a compromised file. Infection typically occurs through file downloads.

3.2.2 Worms

Worms are standalone malicious programs that do not require user interaction and spread rapidly across networks. They can execute payloads to delete files or recruit compromised hosts into botnets—networks of infected machines controlled by a single attacker. Unlike traditional viruses, worms often degrade network performance simply by exhausting available bandwidth. They propagate by scanning systems for exploitable vulnerabilities. In 2022, worms represented the most prevalent malware category. Infection is commonly initiated through phishing emails.

3.2.3 Trojans

Named after the Trojan Horse of Greek mythology, a Trojan is malware disguised as legitimate or harmless software. Once executed, it grants attackers control over the compromised system, enabling data theft, unauthorized network access, botnet enrollment, and host surveillance. Infection typically occurs via malicious email attachments or social engineering.

3.2.4 Keyloggers

Keyloggers are software or hardware tools designed to covertly record keystrokes entered by a user. They capture personal credentials, financial details, and sensitive communications, subsequently exfiltrating them to an attacker.

3.2.5 Ransomware

Ransomware is a rapidly growing class of malware that infiltrates a system, encrypts user files, and demands a ransom payment in exchange for the decryption key. Statistics indicate that the average financial loss resulting from a ransomware incident is approximately 4.65 million USD. Furthermore, the Ransomware-as-a-Service (RaaS) model allows ransomware authors to lease their malware to third-party affiliates, which has significantly accelerated the reach and frequency of attacks.

The most prominent ransomware groups involved in cyber attacks in recent years include:

- Akira: an active ransomware family targeting mid-sized enterprises, first observed in 2023;
- LockBit: an extremely prolific ransomware operation, responsible for over 1,700 attacks in the US alone, generating an estimated 91 million USD from ransom demands in 2020. In 2023, it compromised Royal Mail, demanding an 80 million USD ransom, and later targeted Taiwan Semiconductor Manufacturing Company (TSMC) with a 70 million USD demand;
- Dharma (Crysis variant): operators using this ransomware scan networks for hosts exposing Remote Desktop Protocol (RDP) on TCP port 3389. Once exposed systems are identified, attackers launch brute-force attacks to compromise credentials, gain initial access, and deploy the ransomware payload to encrypt local data;
- MedusaLocker: an aggressive ransomware family operating since 2021. It employs double extortion by encrypting target files and threatening public data leaks; in 2025 alone, it was linked to 430 attacks worldwide;
- Rhysida: a ransomware group known for publishing sensitive corporate and public records, targeting healthcare providers and local government entities;
- ALPHV / BlackCat;
- Royal / BlackSuit;

- RansomHub;
- NoEscape;
- BlackBasta;
- WannaCry.

Chapter 4

Automated Ransomware Analysis

We use a Python script to automate the analysis of a ransomware family chosen from the following:

1. Akira (original version), downloaded from <https://bazaar.abuse.ch/sample/06c2a137c31aae5d02b4d7df61ffd31f1af9a9e59978f15b3f7265cc751bff1f/>;
2. LockBit (version 4.0), downloaded from <https://bazaar.abuse.ch/sample/25fba0e92d00184dde662c7d30aff006851dba296daa6f6f82ce797b66789ed2/>;
3. Rhysida (original version), downloaded from <https://bazaar.abuse.ch/sample/1a9c27e5be8c58da1c02fc4245a07831d5d431cdd1a91cd35d2dd0ad62da71cd/>;
4. MarLock (MedusaLocker variant), downloaded from <https://bazaar.abuse.ch/sample/1bc0575b3fc6486cb2510dac1ac6ae4889b94a955d3eade53d3ba3a92d133281/>;
5. Dharma (Crysis variant), downloaded from <https://bazaar.abuse.ch/sample/e9bbcfb5d9f42ef0dd75eb435e78d5226087679593893e0c08977694e720cd7a/>.

To achieve this, the Cape API is used. At the end of each analysis, a directory is created containing:

- `cape_report.txt`: contains the analysis detections along with the matched YARA rules, dropped files captured during execution, signatures with a brief description and severity level, and TTPs (Tactics, Techniques, and Procedures);

- `files_report.txt`: contains the files read, modified, and deleted within the victim machine during ransomware execution;
- `key_registry_report.txt`: contains the registry keys read, modified, and deleted within the victim machine during ransomware execution;
- `processtree.pdf`: a graph of the process tree generated using the Python "graphviz" library (Digraph);
- `net_capture.pcap`: network traffic captured between the host machine and the victim machine during ransomware execution.

The automated analysis begins by prompting the user to select one of the five ransomware samples:

```
Which ransomware do you want to analyze with Cape?
1. Akira
2. LockBit
3. Rhysida
4. MedusaLocker
5. Dharma
Your choiche:
```

Figure 4.1: Ransomware selection

Once the user makes a choice, the ransomware sample is submitted for analysis using the Cape API endpoint: `http://localhost:8000/apiv2/tasks/create/file/`. A string variable "SAMPLE_FILE" is initialized to store the absolute path to the sample; this string is completed only after the user's input.

```
url_create_file = 'http://localhost:8000/apiv2/tasks/create/file/'
headers = {'Authorization' : 'Token d256567a66cc7cb3f903389f873277b5a1fe4bb2'}
SAMPLE_FILE = '/home/leonald/Scaricati/'
```

Figure 4.2: Initialization of URL, headers, and SAMPLE_FILE

```

Akira = '06c2a137c31aae5d02b4d7df61ffd31f1af9a9e59978f15b3f7265cc751bff1f.zip'
LockBit = '25fba0e92d00184dde662c7d30aff006851dba296daa6f6f82ce797b66789ed2.zip'
Rhysida = '1a9c27e5be8c58da1c02fc4245a07831d5d431cdd1a91cd35d2dd0ad62da71cd.zip'
MedusaLocker = '1bc0575b3fc6486cb2510dac1ac6ae4889b94a955d3eade53d3ba3a92d133281.zip'
Dharma = 'e9bbcfb5d9f42ef0dd75eb435e78d5226087679593893e0c08977694e720cd7a.zip'

```

Figure 4.3: Initialization of substrings used to build the SAMPLE_FILE path

```

while True:
    print('Which ransomware do you want to analyze with Cape?')
    print('1. Akira')
    print('2. LockBit')
    print('3. Rhysida')
    print('4. MedusaLocker')
    print('5. Dharma')

    choix = input('Your choix:')
    if not choix.isdigit():
        print('You must insert a numeric value between 1 and 5.')
        continue

    choix = int(choix)
    if choix < 1 or choix > 5:
        print('You must insert a value between 1 and 5')
        continue
    break

```

Figure 4.4: User selection prompt

```

if choix == 1:
    SAMPLE_FILE += Akira
elif choix == 2:
    SAMPLE_FILE += LockBit
elif choix == 3:
    SAMPLE_FILE += Rhysida
elif choix == 4:
    SAMPLE_FILE += MedusaLocker
else:
    SAMPLE_FILE += Dharma

```

Figure 4.5: Concatenation of the selected sample name to SAMPLE_FILE

Using the "requests" library, a POST request is sent to the API endpoint to start the analysis. Next, the "subprocess" module is used to create a dedicated directory for the selected ransomware, which will store all generated reports. Finally, packet capture is started on the isolated network interface using tcpdump.

```
try:
    #we need 'with' to avoid the use of close() after the read of file opened through open()
    with open(SAMPLE_FILE, 'rb') as sample:
        params = {
            'package' : '',
            'machine' : '',
            'timeout' : 200,
            'options' : 'password=infected',
            'priority' : 2,
            'platform' : '',
            'tags' : '',
            'custom' : '',
            'memory' : '',
            'enforce_timeout' : '',
            'clock' : ''
        }
        multipart_file = {'file' : (SAMPLE_FILE, sample)}
        response = requests.post(url_create_file, headers = headers, files = multipart_file, data = params)
        data = response.json()
        id_task = data['data']['task_ids'][0]
        subprocess.run(['mkdir', f'cape_analysis_{id_task}'], capture_output = True)
        capture = subprocess.Popen(['tcpdump', '-i', 'virbr1', '-w', f'/home/leonal/Scrivania/tirocinio/cape_analysis_
except requests.RequestException as e:
    print(f'request error: {e}')
    exit(1)
except json.JSONDecodeError as e:
    print(f'Not a valid JSON: {e}')
    exit(1)
except FileNotFoundError as e:
    print(f'{SAMPLE_FILE} do not exist: {e}')
    exit(1)
```

Figure 4.6: Starting the analysis, creating the output directory, and launching network capture

From the JSON response returned by the server, the task ID is extracted and used to poll the following status endpoint:

<http://localhost:8000/apiv2/tasks/status>

This endpoint returns the current task processing status, allowing the script to wait until the analysis is complete:

```

while True:
    try:
        status = requests.get(url_task_status, headers = headers).json()['data']
        print('The status of your task with id:', id_task, 'is ' + status)
        if(status != 'reported'):
            print('Please wait...')
            time.sleep(10)
        else:
            break
    except requests.RequestException as e:
        print(f'Request error : {e}')
        exit(1)
    except json.JSONDecodeError as e:
        print(f'Not valid JSON: {e}')
        exit(1)

```

Figure 4.7: Polling loop running until analysis completion

```

Your choiche:5
59
The status of your task with id: 59 is pending
Please wait...

```

Figure 4.8: Terminal output during analysis execution

```

The status of your task with id: 59 is running
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is completed
Please wait...
The status of your task with id: 59 is reported
The analysis is finished.
Reports will be downloaded at /home/leonald/Scrivania/tirocinio/cape_analysis_59

```

Figure 4.9: Analysis completed

Once the analysis finishes, the task ID is used to query two additional API endpoints:

`http://localhost:8000/apiv2/tasks/get/report`

`http://localhost:8000/apiv2/tasks/get/iocs`

The server responses are then stored in the variables `report` and `iocs`.

```
url_get_report = f'http://localhost:8000/apiv2/tasks/get/report/{id_task}'
url_get_iocs = f'http://localhost:8000/apiv2/tasks/get/iocs/{id_task}/detailed'

try:
    response = requests.get(url_get_report, headers = headers)
    report = response.json()
except requests.RequestException as e:
    print(f'Request error : {e}')
    exit(1)
except json.JSONDecodeError as e:
    print(f'Not valid JSON: {e}')
    exit(1)

try:
    response = requests.get(url_get_iocs, headers = headers)
    iocs = response.json()
except requests.RequestException as e:
    print(f'Request error : {e}')
    exit(1)
except json.JSONDecodeError as e:
    print(f'Not valid JSON: {e}')
    exit(1)
```

Figure 4.10: Storing JSON API responses into the "report" and "iocs" variables

To render the process tree, the functions `addNode()` and `addNodeChildren()` are called, and the resulting graph is saved in the previously created directory.

```
#function to add a principal node
def addNode(process = None):
    if process == None:
        return
    else:
        if not process.get('pid') in nodes_pid:
            label = f'{process.get('name')}\nPID: {process.get('pid')}\nNumber of threads: {len(process.get('threads'))}'
            dot.node(f'{process.get('pid')}', label )
            nodes_pid.add(process.get('pid'))
            for child in process['children']:
                addNodeChildren(child,process)
            return
        else:
            return

#function to add a children node
def addNodeChildren(child = None, parent_process = None):
    if not child.get('pid') in nodes_pid:
        child_label = f'{child.get('name')}\nPID: {child.get('pid')}\nPPID: {parent_process.get('pid')}\nNumber of threads: {len(child.get('threads'))}'
        dot.node(f'{child.get('pid')}', child_label)
        nodes_pid.add(child.get('pid'))
        dot.edge(f'{parent_process.get('pid')}', f'{child.get('pid')}')
        for c_child in child.get('children'):
            addNodeChildren(c_child, child)
    else:
        return
```

Figure 4.11: Functions used to build the process tree

```
for process in report['behavior']['processtree']:
    addNode(process)

dot.render(f'cape_analysis_{id_task}/processtree', view = False)
```

Figure 4.12: Using `dot.render()` to export the process tree to the analysis folder

Finally, the three .txt report files are written using the data stored in `report` and `iocs`.

```
try:
    with open(f'cape_analysis_{id_task}/cape_report.txt', 'w') as r:
        r.write('DETECTIONS:\n')
        for detection in report['detections']:
            for detail in detection['details']:
                r.write(f'Cape detected {detection.get('family')} using YARA rule: {detail.get('Yara')}\n')

        r.write('\nDROPPED FILES:\n')
        for i,dropped_file in enumerate(report['dropped'], start = 0):
            r.write(f'NO. {i}:\n')
            r.write(f'Name: {dropped_file.get('name')}\nPath: {dropped_file.get('path')}\n\n')

        r.write('SIGNATURES:\n')
        for i,signature in enumerate(report['signatures'], start = 0):
            r.write(f'NO. {i}:\n')
            r.write(f'Name: {signature.get('name')}\nDescription: {signature.get('description')}\nSeverity level: {signature.get('severity')}\n\n')

        r.write('TTPS:\n')
        for i,ttp in enumerate(report['ttps'], start = 0):
            r.write(f'NO. {i}:\n')
            r.write(f'Signature: {ttp.get('signature')}\nhttps: {ttp.get('ttps')}\n\n')
except Exception as e:
    print(f'Error during report creation: {e}')
```

Figure 4.13: Using `write()` to generate the text report for detections, dropped files, signatures, and TTPs

```
try:
    with open(f'cape_analysis_{id_task}/files_report.txt', 'w') as r:
        r.write('MODIFIED:\n\n')
        for mod in iocs['data']['files']['modified']:
            r.write(f'{mod}\n')

        r.write('\n\nDELETED:\n\n')
        for deleted in iocs['data']['files']['deleted']:
            r.write(f'{deleted}\n')

        r.write('\n\nREAD:\n\n')
        for read in iocs['data']['files']['read']:
            r.write(f'{read}\n')
except Exception as e:
    print(f'Error during File report creation: {e}')
```

Figure 4.14: Using `write()` to generate the text report for modified, deleted, and read files

```

try:
    with open(f'cape_analysis_{id_task}/key_registry_report.txt', 'w') as r:
        r.write('MODIFIED:\n\n')
        for mod in iocs['data']['registry']['modified']:
            r.write(f'{mod}\n')

        r.write('\n\nDELETED:\n\n')
        for deleted in iocs['data']['registry']['deleted']:
            r.write(f'{deleted}\n')

        r.write('\n\nREAD:\n\n')
        for read in iocs['data']['registry']['read']:
            r.write(f'{read}\n')
except Exception as e:
    print(f'Error during Key Registry report creation: {e}')

```

Figure 4.15: Using write() to generate the text report for modified, deleted, and read registry keys

Once execution finishes, the output directory contains all the aforementioned analysis files.



Figure 4.16: Analysis output folder

	cape_report.txt	5,6 kB	Oggi 16:14	☆
	files_report.txt	6,5 kB	Oggi 16:14	☆
	key_registry_report.txt	541,7 kB	Oggi 16:14	☆
	net_capture	34,6 MB	Oggi 16:14	☆
	processtree	2,4 kB	Oggi 16:14	☆
	processtree.pdf	19,3 kB	Oggi 16:14	☆

Figure 4.17: Generated files corresponding to figures 4.12, 4.13, 4.14, and 4.15

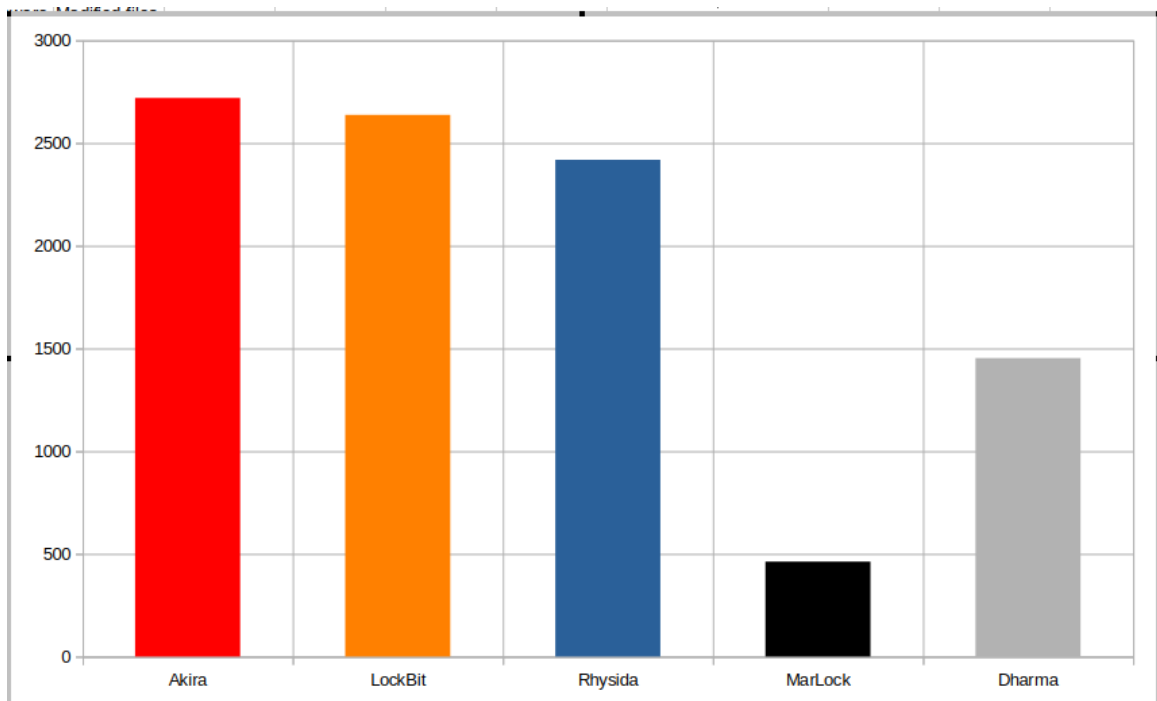


Figure 4.18: Files modified by each ransomware sample

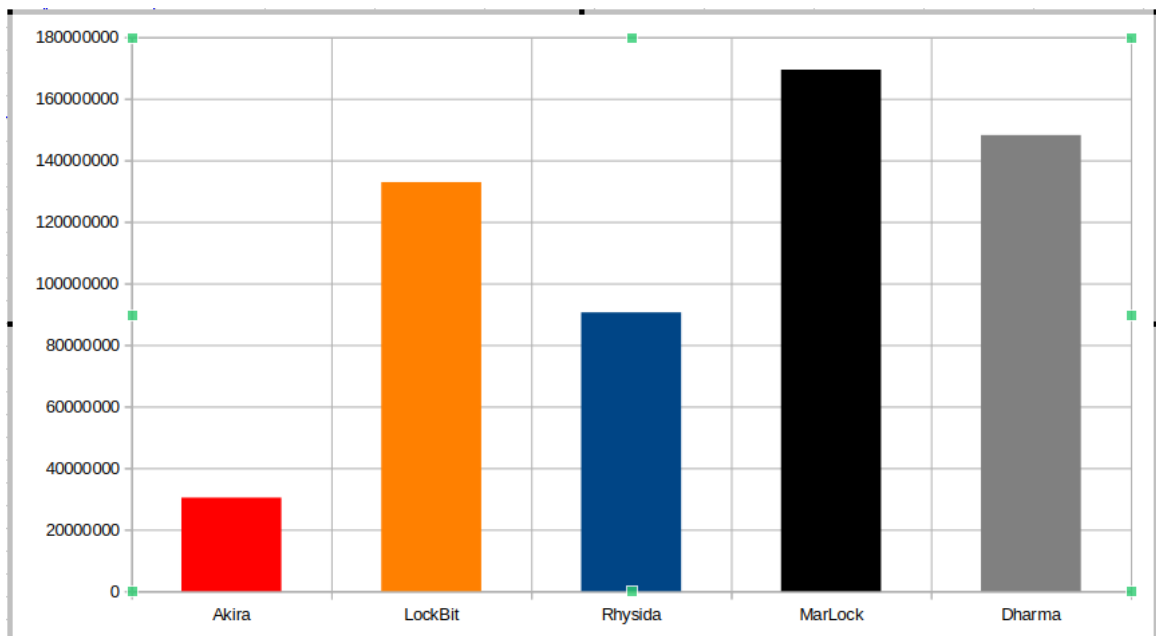


Figure 4.19: Packet bytes exchanged between host and victim during analysis